Project execution and repository governance

Contents

1. Authorized task and stop condition 2

2. Engineering method 2

3. Mandatory writing standards 2

4. Mandatory individual task reports 3

5. PDF deliverables and documentation organization 4

6. Production and destructive-operation boundaries 5

7. Public repository and secrets 5

8. Git and GitHub conventions 5

9. Completion report 6

Applies to this repository and every subdirectory. Project: **HL-CYB-001, Enterprise Zero Trust Security Architecture**.

1. Authorized task and stop condition

Task 001 is limited to planning, documentation, scaffolding, focused documentation checks, safe Git initialization, and reviewed public GitHub publication/synchronization. These actions are already authorized by the owner. Complete that bounded task, then stop for owner review. Do not install software, create VMs, change networks, run security tests, or start an implementation stage during Task 001.

For future work, require **explicit owner approval before each implementation stage A-L** in the implementation guide. Approval must identify the stage, scope, assets, permitted changes, validation, and rollback. Approval of a document, a PR, or a previous stage does not authorize the next stage. Record an approval reference and date without publishing private conversations. A missing approval means dependent implementation does not proceed; read-only planning within the current authorization may continue.

2. Engineering method

1. Work on one bounded task at a time. Read the assignment, architecture, requirements, applicable decisions, and current status before changing anything.
2. Inspect existing files, Git status, remotes, branches, and relevant tooling first. Preserve existing work. Identify conflicts before proposing a reconciliation; do not overwrite or delete owner content without approval.
3. Use documentation-first execution: update the proposed design, measurable outcomes, threat model, affected requirements, test plan, and rollback before implementation.
4. Verify current vendor documentation and version-specific commands. Label proposed commands and identify their target. Never execute a guide merely because it exists.
5. Make the smallest useful change. Do not introduce installation, deployment, attack simulation, remote connectivity, or production access through a validation script or workflow.
6. Use focused automated checks appropriate to the change. Independently verify outcomes through a second observation, such as a policy inspection plus a bounded positive/negative connection test. Re-running the same command is not independent verification. Do not claim independent reviewer sign-off unless someone else actually reviewed it.
7. Separate **engineering verification** from **owner user acceptance testing (UAT)**. A CI pass does not approve business behavior or stage advancement. Record each result, reviewer role, evidence, date, and unresolved issue.
8. Report proposed, implemented, verified, failed, blocked, and accepted states accurately. Attach sanitized evidence to each claimed result. Never turn a planned test into a passing result.
9. Track architectural choices and changes in DECISIONS.md, risks in RISKS.md, requirements/tests in their respective documents, and individual task outcomes in reports/. Keep FINAL-REPORT.md for the project-level final report.

3. Mandatory writing standards

Apply these standards to public READMEs, assignments, implementation guides, architecture/security documents, task reports, final project reports, and user instructions:

- Explain the business problem and intended reader outcome before listing technologies. State what the project does, why it matters, what actually exists, how it was checked, and what evidence supports the outcome.
- Use direct, natural professional English. Prefer concrete actions and short explanations. Remove marketing claims, exaggerated adjectives, stock AI phrases, repeated warnings, and unnecessary decoration.
- Define specialist terms/acronyms on first use where practical, or link the glossary. Keep internal technical language where needed for accuracy. Use consistent project, zone, component, requirement, and stage names.
- Preserve technical commands, configuration values, standards/control mappings, test expectations, and requirement IDs. Improve the explanation around them. Procedures must say what to do, why, how, expected outcome, verification, and recovery; do not replace usable instructions with vague summaries.
- Use links to the authoritative project document instead of repeating large explanations. Keep the README concise and useful to a hiring manager, IT manager, reviewer, or technical reader. Retain the formal assignment, measurable requirements, deliverables, and 100-point rubric.
- Distinguish PLANNED, IN PROGRESS, IMPLEMENTED, VALIDATED, and NOT TESTED as defined in the glossary. Never invent implementation, tests, metrics, screenshots, business results, certifications, independent review, production deployment, or owner UAT.
- Readability work must not edit finalized reports or historical evidence. Record a correction in a new addendum or subsequent task report. Check Markdown links, technical consistency, traceability, public hygiene, and relevant focused tests before publication.

4. Mandatory individual task reports

Every Codex task must create a Markdown-only report during execution and finalize it after the authorized implementation and validation. A task cannot be declared engineering-complete without its corresponding finalized `Ti-Report.md`. Do not generate PDFs for reports or addenda. Use the reporting convention for the required structure and status vocabulary.

- Allocate the next unique sequential number from the existing reports: `reports/T1-Report.md`, `reports/T2-Report.md`, and so on. Check local and remote work before allocation. Do not reserve numbers with empty reports, skip numbers, reuse a number, renumber a report, or start an unapproved next task.
- Use sections A-H: Task Information; Implementation Summary; Testing and Validation; Security and Quality Review; GitHub Operations; Outstanding Issues; Completion Status; Next Task. Include dates, scope/dependencies, changed files/configuration/decisions, executed commands and expected/actual results, PASS/FAIL/NOT TESTED verdicts, risks/remediation, branch/commit/PR/CI/merge/synchronization, engineering/independent verification/owner UAT, and the next bounded task with its approval requirements.
- Start with `Report state: DRAFT`. Finalize only after reviewing the actual evidence; change the marker to `Report state: FINALIZED` and record the evidence cutoff and finalization date. Missing tests are NOT TESTED. Owner UAT remains PENDING unless the owner explicitly confirms acceptance; record that confirmation reference before marking UAT PASS.
- A finalized report is an immutable historical record. Never overwrite, edit, renumber, or delete it, including for later readability changes. Record corrections or follow-up evidence in a separately numbered addendum such as `reports/T1-Addendum-001-Reporting-Governance.md`, or in a subsequent authorized task report. An addendum has the same A-H structure and state marker, identifies its parent task, and does not allocate a new task number.

- The Task 001 reporting migration is an authorized Task 001 addendum, not Task 2. Its historical report is T1-Report.md; the original source and evidence are preserved there. Later tasks use the next available number only after the owner authorizes their scope.

- Keep all individual task reports and report addenda in `reports/`, in Markdown only. Exclude the entire directory, including its index, from PDF generation. Existing T3-era PDFs may remain as byte-frozen archives; do not regenerate them or create new report PDFs. Keep raw evidence private and link only reviewed public artifacts. The project-level final report is not an individual task log and still requires its document PDF.

- Final reports describe a named evidence cutoff. Do not invent the report commit's own SHA, future CI result, merge outcome, synchronization, independent review, or owner acceptance. After publishing a report, verify its PR/CI/merge and local/remote state and give those observed results in the final response; later written corrections require a new addendum.

- Run `python scripts/validate_docs.py --require-finalized` before declaring completion or merging the completed task. The checks cover report numbering, required sections, and preservation of committed finalized versions. They support review; they do not supply owner acceptance or enforce GitHub branch protection.

5. PDF deliverables and documentation organization

- Keep editable documents in the approved purpose folders under `docs/`: assignments, lab-guides, architecture, security, testing, governance, final-report, and related guide/index locations. Keep README.md and AGENTS.md at the root; code/configurations/tests/evidence stay in their designated folders.

- Markdown is the source of truth. Generate adjacent PDFs for maintained user-facing documents outside `reports/` using the build guide. Assignment, lab guide, architecture, security, testing/acceptance, and the project final report retain PDF editions. Never edit a PDF independently or change a finalized report's Markdown. Compatibility redirects are navigation aliases, not duplicate source editions.

- Use `python scripts/build_docs.py` with the pinned documentation dependencies. Rebuild changed sources, then run `python scripts/build_docs.py --check --rebuild-check`, documentation/report checks, focused tests, and manual publication review. Do not publish unknown binary artifacts or hand-edit the generated manifest to suppress drift.

- Preserve searchable text, tables/code, the diagram's actual topology, hyperlinks, numbered sections, document version, and page navigation. Render and inspect representative title/contents/table/diagram/code/page-break pages; extraction alone cannot establish layout quality.

- Keep PDFs deterministic: fixed metadata, source-derived versions, embedded package fonts, and unchanged-output skipping. Commit reviewed sources, generated PDFs, manifest, and relevant build/CI changes. No-change builds must avoid binary churn.

- Record applicable document PDF generation, source/frozen-history comparisons, structural/text/link checks, visual review, and actual CI/merge state in the Markdown-only individual task report. The project final report must remain visibly a template until its real implementation/evidence exists. PDF generation never authorizes lab execution or owner UAT.

6. Production and destructive-operation boundaries

Use synthetic business data, separate lab identities, separate remote-access environments, and isolated lab resources. No production trust, federation, identity synchronization, credential reuse, corporate Microsoft 365 connection, PE-DAP source code, customer data, corporate website access for testing, or production hosting integration is permitted.

No destructive operation, disk installation/formatting, trust creation, route publication, firewall change, exposure to the Internet, or production operation without explicit authorization covering that action. Proxmox installation must never select a disk until the owner has approved its exact identity and backup/recovery plan. If unexpected connectivity or possible production impact appears, stop the affected operation, isolate the lab using the approved recovery procedure, and report facts.

No security probes, scans, password attacks, malware, exploit execution, or attack traffic outside an explicitly approved lab scope. Home and production isolation is verified through configuration inspection, passive observation, and lab-owned mock protected targets; never actively probe real protected networks to demonstrate a deny rule.

7. Public repository and secrets

Publish only reviewed public-safe artifacts. Keep tokens, passwords, private keys, lab inventories and actual addresses, corporate details, raw evidence, logs, packet captures, VM images, state files, and backups outside the public repository. `.gitignore` is not a security boundary and does not remove already tracked files. Never store authentication tokens in project files, Git URLs, command arguments, screenshots, or commit messages. Use GitHub CLI/Git Credential Manager or another supported secure credential store.

Before every publication, inspect the complete proposed tracked set, staged diff, filenames, and new history; run the validator and relevant checks. Resolve all findings and perform a manual sensitive-data review. Do not bypass scanners to publish a finding. Synthetic examples must use reserved names or symbolic address variables. If a secret is exposed, stop publication, revoke/rotate it using the owner's authorized incident process, preserve private incident evidence, and obtain approval for any history rewrite.

Never select a software license without owner approval.

8. Git and GitHub conventions

- Authorized repository: waqashamdani-ca/enterprise-zero-trust-security, **PUBLIC**, default branch `main`.
- Verify the authenticated identity and repository permissions before writing. Check both author and committer use the GitHub noreply identity for new public commits, including merges. Local Git identity does not establish the email used by a GitHub web/API-created commit. Never alter global Git configuration or account privacy settings silently.
- Check whether the remote exists before creating it. Inspect existing branches and contents, reconcile changes on a branch, and preserve history. Never force-push or overwrite existing work without explicit owner approval.
- The initial reviewed Task 001 bootstrap may initialize `main` directly in a new empty repository. Future engineering uses task branches, for example `task/002-hardware-readiness`.
- Begin from clean `main` and `git pull --ff-only origin main`. Stop on divergence; do not use an automatic merge, destructive reset, or forced update to conceal differences.
- Stage explicit public-safe paths. Make focused commits with meaningful messages, push the task branch, run applicable CI/security checks, and use a PR for review before merging into `main`.

- PRs describe the problem, resulting behavior, scope, affected requirement/test IDs, validation evidence, risks, rollback, and applicable owner approval. Stage approval and owner UAT remain distinct records.
- After merge, fast-forward local `main`. Confirm `git remote -v`, `git status`, `git branch -vv`, `git log -1`, and `git ls-remote origin`; verify the local and remote commit IDs agree.
- Where repository rules permit, a normal local merge with the verified noreply author/committer can be pushed after exact-head PR checks/review requirements pass. Confirm GitHub records the PR as merged. Never use this method to bypass a required approval, check, or protection. If a permitted merge method cannot preserve the required publication identity, stop that operation for an owner decision.
- Synchronization is intentional Git activity. Never enable background upload or treat all local files as publishable.
- Workflows use least privilege (`contents: read` unless a reviewed task requires more), commit-pinned official actions, no persistent checkout credentials, no secrets in fork PR checks, and no lab access. Do not use `pull_request_target` to execute untrusted PR code.
- Repository rulesets, required checks, reviewer settings, and licensing require explicit owner decisions; do not silently change them.

9. Completion report

Link the finalized individual task report and any authorized addendum. State changed files, checks actually run, local/remote commit IDs, repository URL, visibility/default branch, PR/merge and CI results, owner UAT status, unresolved decisions/risks, and exactly one recommended next task. Stop at the authorized task boundary.